Docker-03

1. Create a image from running container.

1. Objective

Create an Nginx container, customize its default webpage, create a new Docker image from the modified container, and verify the image by running another container.

2. Environment details

Component	Value
Server	AWS EC2 Ubuntu
Base image	`nginx:latest`
Original container	`docker03-nginx`
Original host port	8084
Custom image	`docker03-nginx:v1`
New container	`docker03-nginx-test`
New host port	8085

Custom image ID	`20f05900ccda`
Result	SUCCESS

3. Implementation steps

Step 1 — Check existing Docker containers

```
sudo docker ps -a
```

Purpose: Check container status and avoid existing container-name or port conflicts.

Step 2 — Create Nginx container

```
sudo docker run -d \
  --name docker03-nginx \
  -p 8084:80 \
  nginx:latest
```

Purpose: Create and run Nginx on host port 8084.

Step 3 — Verify container

```
sudo docker ps
curl -I http://localhost:8084/
```

Actual result:

```
HTTP/1.1 200 OK
```

Step 4 — Create a customized HTML page

```
cat > docker03-index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Docker-03 Task 1</title>
</head>
<body>
```

```
    <h1>Welcome to Docker-03</h1>
    <h2>Custom Nginx Container</h2>
    <p>Created by Chandrashekar</p>
    <p>This webpage was customized inside a running container.</p>
</body>
</html>
EOF
```

Step 5 — Copy HTML into the running container

```
sudo docker cp docker03-index.html \
  docker03-nginx:/usr/share/nginx/html/index.html
```

Purpose: Replace the default Nginx welcome page.

Verify:

```
curl http://localhost:8084/
```

Step 6 — Create Docker image using `docker commit`

```
sudo docker commit \
  docker03-nginx \
  docker03-nginx:v1
```

Purpose: Create a reusable image containing the modified webpage.

Step 7 — Verify the newly created image

```
sudo docker images
```

Actual result:

```
IMAGE                IMAGE ID
docker03-nginx:v1    20f05900ccda
```

Step 8 — Run another container using the new image

```
sudo docker run -d \
  --name docker03-nginx-test \
  -p 8085:80 \
  docker03-nginx:v1
```

Step 9 — Verify deployment

```
sudo docker ps
curl http://localhost:8085/
```

Actual output included:

```
<h1>Welcome to Docker-03</h1>
<h2>Custom Nginx Container</h2>
<p>Created by Chandrashekar</p>
```

This confirms the committed image contains the customized HTML file.

4. Browser verification

To verify the website through your browser, use your EC2 instance's current public IP.

Original Nginx container — Port 8084

```
http://<EC2-PUBLIC-IP>:8084/
```

Committed image container — Port 8085

```
http://<EC2-PUBLIC-IP>:8085/
```

Both URLs should display the customized webpage.

For example, your browser should show:

EC2 Public IP : 8085

Welcome to Docker-03

Custom Nginx Container

Created by Chandrashekar

This webpage was customized inside a running container.

If the page doesn't open externally, verify that the EC2 security group allows inbound TCP traffic on ports 8084 and 8085 from your IP address.

5. Docker Hub verification — Optional

Important: You have successfully created the image locally, but you have not pushed `docker03-nginx:v1` to Docker Hub yet.

If you want to store the committed image in Docker Hub as additional practice, follow these steps.

Step 1 — Tag the image

```
sudo docker tag docker03-nginx:v1 \
  chandrashekar8639211489/docker03-nginx:v1
```

Step 2 — Login to Docker Hub

```
sudo docker login -u chandrashekar8639211489
```

Use your Docker Hub access token when prompted. Do not share it.

Step 3 — Push the image

```
sudo docker push \
  chandrashekar8639211489/docker03-nginx:v1
```

Step 4 — Verify on Docker Hub

Open Docker Hub and navigate to My Hub → Repositories → docker03-nginx → Tags.

Confirm that the `v1` tag appears. This step is complete only after the push succeeds and the repository shows the image.

6. Important interview questions

Q1. What is `docker commit`?

`docker commit` creates a new Docker image from the filesystem changes in an existing container.

Q2. What is the difference between `docker commit` and `docker build`?

`docker commit` captures changes made in a container, whereas `docker build` creates an image from Dockerfile instructions.

Q3. Why do we prefer Dockerfiles in production?

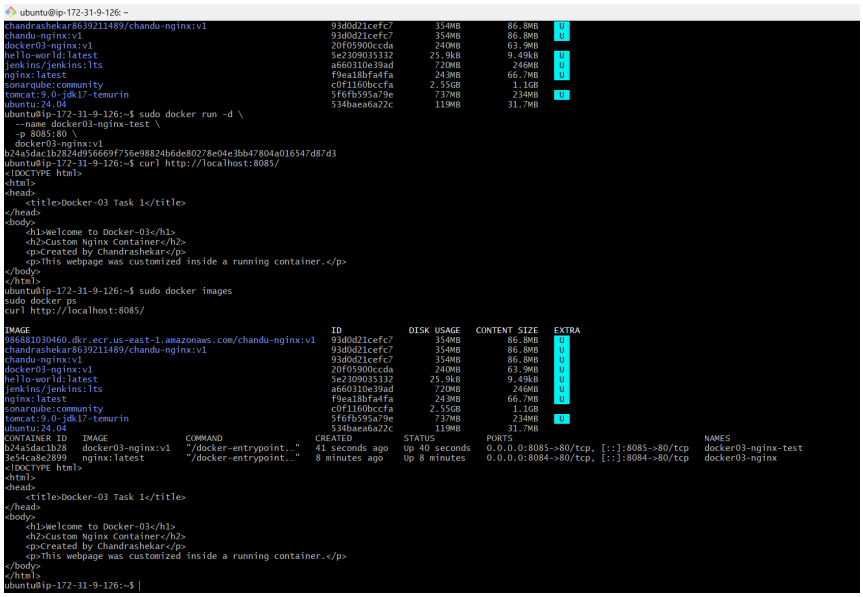
Dockerfiles provide repeatable, version-controlled image builds, making them easier to maintain, review, and automate in CI/CD pipelines.

Q4. Does `docker commit` include volume data?

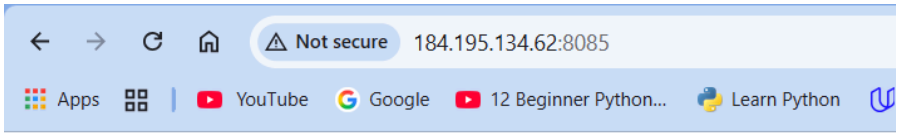
No. Data stored in mounted Docker volumes is not included in the committed image.

7. Final conclusion

Successfully created an Nginx container, customized the webpage, generated a Docker image using `docker commit`, started a new container using that image, and verified the application on port 8085.







Welcome to Docker-03

Custom Nginx Container

Created by Chandrashekar

This webpage was customized inside a running container.

2. Copy image from local machine to docker server and load the image.

1. Objective

Transfer a Docker image from a local machine to a remote Docker server using `docker save`, `scp`, and `docker load`, without requiring the remote server to download the image from Docker Hub.

2. Environment details

Component	Details
Source machine	Windows — Docker Desktop
Docker version	29.8.0
Destination server	AWS EC2 — Ubuntu

Docker version on EC2	29.9.0
Docker image	`alpine:latest`
Image ID	`294b683cb724`
TAR filename	`alpine-docker03.tar`
TAR file size	3.8 MB
Transfer method	SCP (SSH)
Task status	SUCCESS

3. Implementation steps

Step 1 — Verify Docker on Windows

Opened Windows Git Bash and executed:

```
docker --version
docker images
```

Purpose: Verify Docker Desktop installation and check available local Docker images.

Step 2 — Pull Alpine image

```
docker pull alpine:latest
```

Purpose: Download the Alpine Linux Docker image from Docker Hub.

Verification:

docker images

Actual image:

alpine:latest   294b683cb724

Step 3 — Export Docker image as TAR

docker save -o alpine-docker03.tar alpine:latest

Purpose: Save the Docker image as a transferable TAR archive.

Verify:

ls -lh alpine-docker03.tar

Actual output:

-rw-r--r-- 1 HP 197121 3.8M alpine-docker03.tar

Step 4 — Transfer TAR file to EC2 using SCP

Executed from Windows Git Bash:

```
scp -i "my_server.pem" \
  alpine-docker03.tar \
  ubuntu@184.195.134.62:/home/ubuntu/
```

Purpose: Securely transfer the image archive from Windows to the Ubuntu EC2 Docker server.

Actual output:

alpine-docker03.tar 100% 3866KB

The SCP transfer completed successfully.

Step 5 — Verify transferred file on Ubuntu

Logged in to the Ubuntu EC2 server.

ls -lh /home/ubuntu/alpine-docker03.tar

Actual result:

-rw-r--r-- 1 ubuntu ubuntu 3.8M alpine-docker03.tar

Purpose: Confirm the TAR file was successfully transferred to the destination server.

Step 6 — Remove existing Alpine image

Alpine had previously been downloaded directly onto EC2. To demonstrate the image import process, removed the existing image:

sudo docker rmi alpine:latest

Actual output:

Untagged: alpine:latest
Deleted: sha256:294b683c...

Verify:

sudo docker images alpine

Result: No Alpine image was listed.

Step 7 — Load Docker image from TAR

sudo docker load -i /home/ubuntu/alpine-docker03.tar

Actual output:

Loaded image: alpine:latest

Purpose: Import the transferred Docker image into the EC2 Docker image store.

Step 8 — Verify loaded Docker image

sudo docker images alpine

Actual output:

IMAGE ID
alpine:latest 294b683cb724

Step 9 — Create and test a container

```
sudo docker run --rm alpine:latest \
  echo "Docker-03 Task 2 Successful"
```

Actual output:

```
Docker-03 Task 2 Successful
```

Result: Docker successfully created and executed a container using the transferred image.

4. Issues faced and resolutions

Issue	Root cause	Resolution
Docker permission denied	Ubuntu user couldn't access Docker daemon socket	Used `sudo docker`
PEM key not found	Entered `my-server.pem` instead of `my_server.pem`	Corrected key filename
SSH hostname error	Used `EC2_` prefix before public IP	Used the actual IP address
SCP permission denied	Existing destination TAR file was owned by root	Renamed old TAR file and transferred again

The command used to preserve the existing root-owned TAR file was:

```
sudo mv /home/ubuntu/alpine-docker03.tar \
 /home/ubuntu/alpine-docker03-old.tar
```

After that, SCP succeeded.

5. Important commands and purposes

Command	Purpose

`docker pull`	Download image from registry
`docker images`	List available images
`docker save -o`	Export image to TAR archive
`scp -i`	Transfer file securely using SSH key
`ls -lh`	Verify file and size
`docker rmi`	Remove local Docker image
`docker load -i`	Import Docker image from TAR
`docker run --rm`	Run container and remove it automatically afterward

6. Real-time DevOps interview questions

Q1. How do you transfer Docker images between servers?

I use `docker save` to export the image into a TAR file, transfer it using SCP, and import it using `docker load`.

Q2. Why use `docker save` and `docker load`?

These commands are useful for transferring Docker images to restricted or offline environments where directly pulling from Docker Hub is not possible.

Q3. What is the difference between `docker save` and `docker export`?

`docker save` creates an archive of Docker images, including their image layers and tags. `docker export` exports the filesystem of a container without preserving image history in the same way.

Q4. Why did SCP return permission denied?

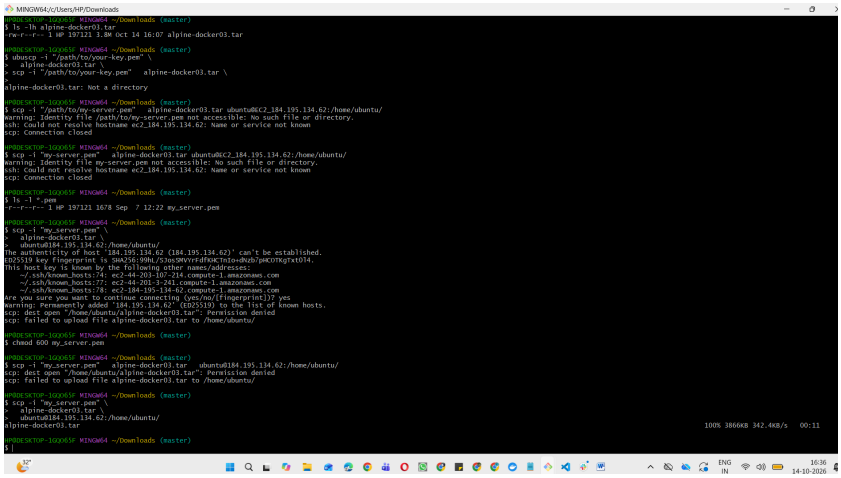
An existing destination file was owned by root and could not be overwritten by the Ubuntu user. I resolved it by renaming the old file and copying the new archive.

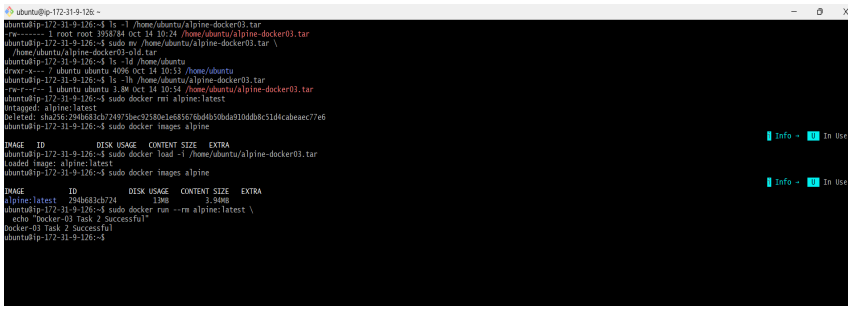
Q5. Is internet access required for `docker load`?

No. Once the TAR archive is available locally, Docker can load the image without connecting to an external container registry.

7. Conclusion

Successfully downloaded Alpine Linux on Windows Docker Desktop, exported the image as a TAR archive, securely transferred it to AWS EC2 using SCP, imported the image with `docker load`, and verified successful container execution.





3. Create Docker image using alpine and customize with tomcat.

1. Objective

Create a lightweight, customized Tomcat Docker image using Alpine Linux, install Java 17 and Apache Tomcat 9, deploy a sample WAR application, and verify that the application is accessible.

2. Environment details

Component	Details
Server	AWS EC2 — Ubuntu
Base image	`alpine:3.21`
Java	OpenJDK 17 JRE
Tomcat	Apache Tomcat 9.0.102

Application	`sample.war`
Custom image	`alpine-tomcat:v1`
Container	`alpine-tomcat-task3`
Host port	8086
Container port	8080
Final result	SUCCESS

3. Implementation steps

Step 1 — Create the project directory

Logged in to the Ubuntu EC2 server as the root user.

mkdir -p ~/docker03-task3

cd ~/docker03-task3

pwd

Actual working directory:

/root/docker03-task3

Purpose: Store the Dockerfile and application WAR file in one project directory.

Step 2 — Verify Alpine image

```
docker images alpine
```

The Alpine image transferred during Task 2 was available locally.

Step 3 — Download the sample WAR application

```
curl -fL \
  https://tomcat.apache.org/tomcat-9.0-doc/appdev/sample/sample.war \
  -o sample.war
```

Verify:

```
ls -lh sample.war
file sample.war
```

Actual output:

```
-rw-r--r-- 1 root root 4.5K sample.war
sample.war: Java archive data (JAR)
```

Purpose: Download a sample Java application for deployment in Tomcat.

Step 4 — Create the Dockerfile

```
cat > Dockerfile <<'EOF'
FROM alpine:3.21

LABEL maintainer="Chandrashekar"

RUN apk add --no-cache openjdk17-jre curl tar

ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk
ENV CATALINA_HOME=/opt/tomcat

WORKDIR /opt/tomcat

RUN curl -fSL \
```

```
https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.102/bin/apache-tomcat-9.0.102.tar.gz \
    -o /tmp/tomcat.tar.gz \
    && tar -xzf /tmp/tomcat.tar.gz -C /opt/tomcat --strip-components=1 \
    && rm /tmp/tomcat.tar.gz

COPY sample.war /opt/tomcat/webapps/sample.war

EXPOSE 8080

CMD ["/opt/tomcat/bin/catalina.sh", "run"]
EOF
```

Step 5 — Understand Dockerfile instructions

Instruction	Purpose
`FROM`	Use Alpine Linux as the base image
`LABEL`	Add maintainer information
`RUN apk add`	Install Java and required utilities
`ENV`	Define environment variables
`WORKDIR`	Set the working directory
`RUN curl`	Download Apache Tomcat

`tar -xzf`	Extract the downloaded Tomcat archive
`COPY`	Copy the WAR into Tomcat's deployment directory
`EXPOSE 8080`	Document the Tomcat container port
`CMD`	Start Apache Tomcat in the foreground

Step 6 — Verify project files

ls -lh

cat Dockerfile

Actual files:

Dockerfile

sample.war

Step 7 — Build the customized Docker image

docker build -t alpine-tomcat:v1 .

Purpose: Build a Docker image using the Dockerfile.

Command explanation:

- `docker build`: Build a Docker image.
- `-t`: Assign an image name and tag.
- `alpine-tomcat`: Image name.
- `v1`: Image version.
- `.`: Current directory as the build context.

Actual result:

[+] Building 228.0s (11/11) FINISHED

naming to docker.io/library/alpine-tomcat:v1

Build status: SUCCESS

Step 8 — Verify the Docker image

```
docker images alpine-tomcat
```

Actual output:

```
IMAGE              ID             DISK USAGE   CONTENT SIZE
alpine-tomcat:v1   28ea85f043ff   313MB        84.9MB
```

Step 9 — Create and run the Tomcat container

```
docker run -d \
  --name alpine-tomcat-task3 \
  -p 8086:8080 \
  alpine-tomcat:v1
```

Purpose: Create a Tomcat container from the customized image.

The command maps host port 8086 to Tomcat's internal port 8080.

Step 10 — Verify the running container

```
docker ps
```

Actual output included:

```
IMAGE          alpine-tomcat:v1
STATUS          Up
PORTS           0.0.0.0:8086->8080/tcp
NAMES           alpine-tomcat-task3
```

Step 11 — Verify application deployment

```
curl -I http://localhost:8086/sample/
```

Actual result:

```
HTTP/1.1 200
Content-Type: text/html
Content-Length: 636
```

Result: The sample Java application was deployed successfully.

4. Browser verification

To access the application externally, open:

http://<EC2-PUBLIC-IP>:8086/sample/

Ensure the EC2 security group allows inbound TCP port `8086` from your IP address.

Application verification

Server-side HTTP 200 was verified. External browser access is an additional check.

Sample WAR deployment successful

5. Image size comparison

Compared the customized image with the official Tomcat image previously downloaded.

Docker image	Disk usage	Content size
Official Tomcat (JDK 17 Temurin)	737 MB	234 MB
Custom Alpine Tomcat	313 MB	84.9 MB

Image content size comparison

Observed Docker-reported content sizes, MB

0 MB65 MB130 MB195 MB260 MBOfficial TomcatAlpine Tomcat

The custom image had approximately 64% less reported content size. These images used different Tomcat/base-image versions, so the comparison is illustrative rather than strictly like-for-like.

6. Important Docker commands

Command	Purpose

`docker images`	List local Docker images
`docker build -t`	Build and tag an image
`docker run -d`	Start a container in background
`docker ps`	Display running containers
`docker logs`	Check container application logs
`curl -I`	Verify HTTP response headers
`docker exec -it`	Execute commands inside a container
`docker inspect`	Inspect container or image configuration

7. Real-time DevOps interview questions

Q1. Why did you use Alpine Linux for your Tomcat image?

Alpine Linux provides a lightweight base image. I installed only the packages required by the application, helping reduce the final image size.

Q2. What is the purpose of `WORKDIR`?

`WORKDIR` sets the working directory for subsequent Dockerfile instructions and the default working directory when the container starts.

Q3. What is the difference between `EXPOSE` and `-p`?

`EXPOSE` documents the container's intended listening port. The `-p` option publishes a container port on the host.

Q4. Why do we use `CMD ["catalina.sh", "run"]`?

It starts Tomcat in the foreground, allowing Tomcat to remain the main application process inside the container.

Q5. What happens if the WAR file is missing?

Tomcat may still start, but the expected application will not be deployed at `/sample/`. The application URL may return HTTP 404.

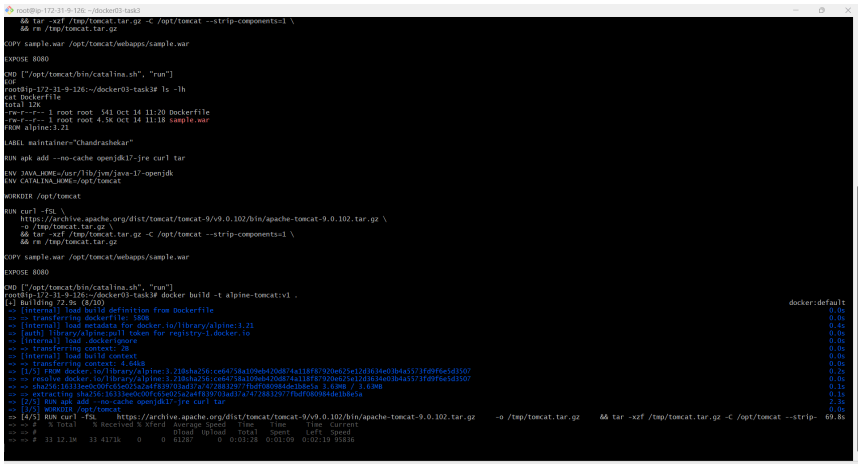
Q6. How did you verify the application deployment?

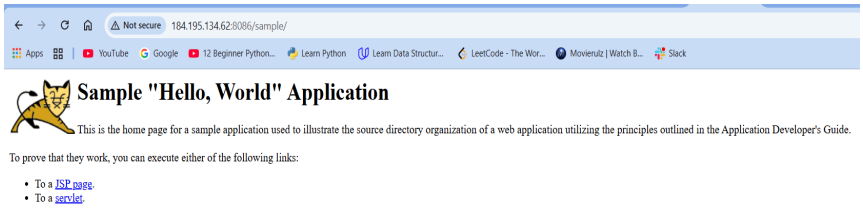
I checked the running container using `docker ps` and executed `curl -I http://localhost:8086/sample/`, which returned HTTP 200.

8. Conclusion

Successfully created a custom Apache Tomcat Docker image using Alpine Linux, installed OpenJDK 17, deployed a Java WAR application, started the container on port 8086, and verified the deployment with HTTP 200.

Final Result: Docker-03 Task 3 — SUCCESS





4. Create single stage and multi stage docker file using this source code: https://github.com/betawins/multi-stage-example.git

1. Objective

Build a Java Spring Boot application using two Dockerfile approaches:

- Single-stage: Compile and run the application in the same image.
- Multi-stage: Build the application in one stage and run it in a separate, smaller runtime image.

Deploy both applications, verify their health endpoints, and compare image sizes.

2. Environment details

Component	Details
Server	AWS EC2 Ubuntu

Source code	betawins/multi-stage-example
Application	Spring Boot 2.2.6.RELEASE
Java	Java 8
Build tool	Maven Wrapper
Output	Executable JAR
Single-stage port	8087
Multi-stage port	8088
Final status	SUCCESS

3. Implementation steps

Step 1 — Create project directory

Logged in to the Ubuntu EC2 server and executed:

```
cd /root
mkdir -p docker03-task4
cd docker03-task4
```

Purpose: Create a separate workspace for Docker-03 Task 4.

Step 2 — Clone GitHub repository

```
git clone https://github.com/betawins/multi-stage-example.git
```

```
cd multi-stage-example
```

Purpose: Download the Spring Boot application source code.

Step 3 — Verify project files

```
pwd
ls -la
cat Dockerfile
cat pom.xml
```

The repository contained `pom.xml`, `mvnw`, `src`, and an existing Dockerfile.

The Maven configuration specified Spring Boot `2.2.6.RELEASE` and Java `1.8`.

4. Single-stage Dockerfile implementation

Step 4 — Create Dockerfile.single

```
cat > Dockerfile.single <<'EOF'
FROM eclipse-temurin:8-jdk

LABEL maintainer="Chandrashekar"

WORKDIR /app

COPY . .

RUN chmod +x mvnw && ./mvnw clean package -DskipTests

EXPOSE 8080

CMD ["java", "-jar", "/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
EOF
```

Purpose: Build and execute the application using a single Java JDK image.

Step 5 — Build single-stage image

```
docker build -f Dockerfile.single \
  -t springboot-single:v1 .
```

Actual result:

```
[+] Building 41.4s (10/10) FINISHED
naming to docker.io/library/springboot-single:v1
```

Build status: SUCCESS

Step 6 — Run single-stage container

```
docker run -d \
  --name springboot-single-container \
  -p 8087:8080 \
  springboot-single:v1
```

Purpose: Deploy the Spring Boot application on host port 8087.

Step 7 — Verify single-stage application

```
curl -i http://localhost:8087/actuator/health
```

Actual output:

```
HTTP/1.1 200
Content-Type: application/vnd.spring-boot.actuator.v3+json

{"status":"UP"}
```

The application was also verified successfully through the browser.

5. Multi-stage Dockerfile implementation

Step 8 — Create Dockerfile.multi

```
cat > Dockerfile.multi <<'EOF'
# Stage 1: Build application
FROM eclipse-temurin:8-jdk AS builder

WORKDIR /app
```

```
COPY . .

RUN chmod +x mvnw && ./mvnw clean package -DskipTests

# Stage 2: Run application
FROM eclipse-temurin:8-jre

WORKDIR /app

COPY --from=builder \
  /app/target/multi-stage-example-0.0.1-SNAPSHOT.jar \
  /app/app.jar

EXPOSE 8080

CMD ["java", "-jar", "/app/app.jar"]
EOF
```

Purpose: Separate the application build environment from the runtime environment.

The first stage compiles the Java application. The second stage copies only the generated JAR into the JRE-based runtime image.

Step 9 — Build multi-stage image

```
docker build -f Dockerfile.multi \
  -t springboot-multi:v1 .
```

The image was built successfully.

Step 10 — Verify both Docker images

```
docker images springboot-single
docker images springboot-multi
```

Actual results:

```
springboot-single:v1   639MB disk usage   201MB content
springboot-multi:v1    437MB disk usage   124MB content
```

Step 11 — Run multi-stage container

```
docker run -d \
  --name springboot-multi-container \
  -p 8088:8080 \
  springboot-multi:v1
```

Step 12 — Verify multi-stage application

```
curl -i http://localhost:8088/actuator/health
```

Actual output:

```
HTTP/1.1 200
Content-Type: application/vnd.spring-boot.actuator.v3+json


{"status":"UP"}
```

Result: The multi-stage Spring Boot application was deployed successfully.

6. Browser verification

Both applications were verified using the EC2 public IP.

Single-stage application

http://184.195.134.62:8087/actuator/health

Multi-stage application

http://184.195.134.62:8088/actuator/health

Both screenshots showed:

```
{
  "status": "UP"
}
```

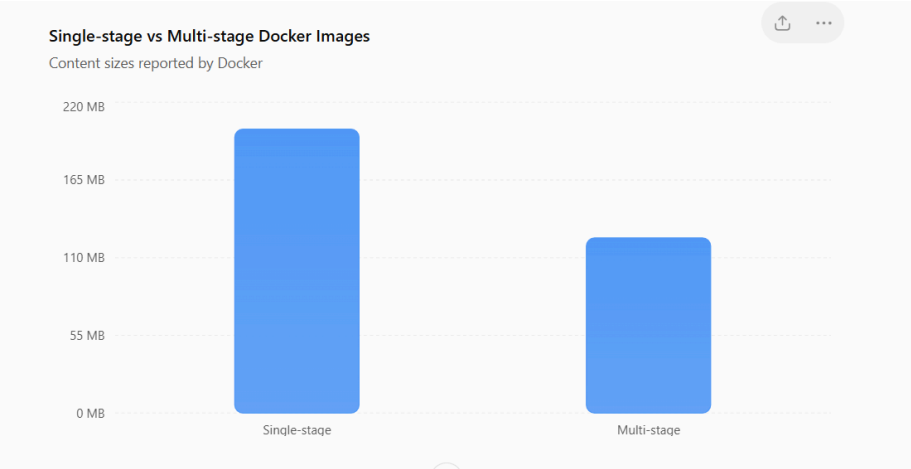
If these URLs are no longer available later, check that the EC2 instance and containers are running and the public IP has not changed.

7. Image size comparison

Single-stage vs Multi-stage Docker Images

Content sizes reported by Docker

0 MB55 MB110 MB165 MB220 MBSingle-stageMulti-stage



Metric	Single-stage	Multi-stage
Docker disk usage	639 MB	437 MB
Image content size	201 MB	124 MB
Java runtime	JDK 8	JRE 8
Build environment in final image	Included	Excluded
Application health	UP	UP

Content size saved

77 MB

Content size reduction

38.3%

The multi-stage image was smaller because the final image did not contain the build-stage source files and build tools.

8. Important commands used

Command	Purpose
`git clone`	Download source code
`cat pom.xml`	Inspect Maven configuration
`docker build -f`	Build using a specific Dockerfile
`docker images`	Check image sizes
`docker run -d`	Start a container in background
`-p 8087:8080`	Publish the container port on the host
`docker ps`	Check running containers
`docker logs`	Troubleshoot application errors

`curl -i`	Verify HTTP status and response
`COPY --from=builder`	Copy artifacts between build stages

9. Real-time DevOps interview questions

Q1. What is a single-stage Dockerfile?

A single-stage Dockerfile uses one `FROM` instruction to build and run the application in a single image.

Q2. What is a multi-stage Dockerfile?

A multi-stage Dockerfile uses multiple build stages. It allows us to build an application in one stage and copy only required artifacts into the final runtime image.

Q3. Why do we use multi-stage Dockerfiles?

To reduce final image size, exclude unnecessary build tools and source files, and potentially reduce security risks.

Q4. What is the purpose of `COPY --from=builder`?

It copies files from a previous build stage into the current stage. In this task, it copied the generated Spring Boot JAR into the runtime image.

Q5. What happens if the build stage fails?

The Docker build fails, so a new final image is not produced successfully.

Q6. How did you verify your application?

I used the Spring Boot Actuator endpoint `/actuator/health`. Both applications returned HTTP 200 with `{"status":"UP"}`.

Q7. How much image size did you reduce?

In my practical, the Docker-reported content size decreased from 201 MB to 124 MB, a reduction of approximately 38.3%.

Q8. Why use a JRE instead of a JDK in the final stage?

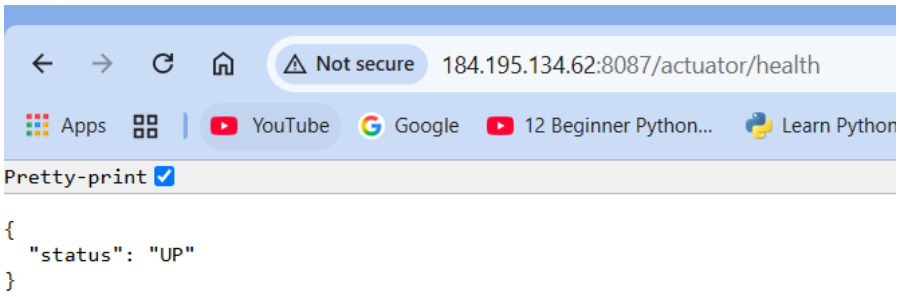
The JDK is needed to compile Java code, whereas a JRE provides the runtime required to execute the application. Using a JRE-based final image can reduce unnecessary build dependencies.

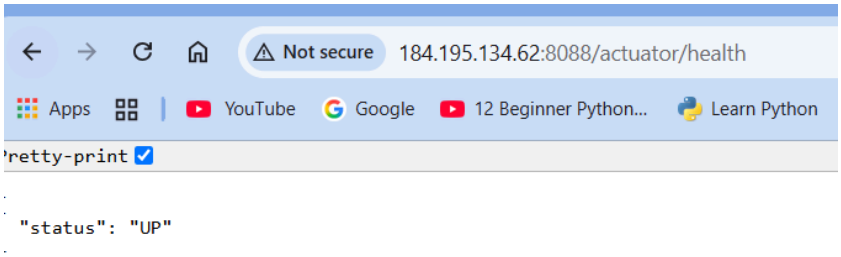
10. Conclusion

Successfully implemented single-stage and multi-stage Docker builds using a Spring Boot Maven application. Both containers were deployed, tested locally and verified in the browser.

The multi-stage implementation reduced the reported content size by approximately 38.3%, while both applications returned healthy responses.

Final Result: Docker-03 Task 4 — SUCCESS





5. Install docker compose and execute sample application.

1. Objective

Deploy a multi-container application using Docker Compose, configure persistent volumes and networking, and verify communication between WordPress and MySQL.

The purpose of this task is to understand how DevOps engineers manage multiple interconnected containers using a single YAML configuration file.

2. Environment details

Component	Actual configuration
Server	AWS EC2 — Ubuntu
Docker Compose	v5.6.0
Application	WordPress
Database	MySQL 8.0
WordPress image	`wordpress:php8.3-apache`
MySQL image	`mysql:8.0`
Host port	8089
Database port	3306 (internal)
Network	User-defined bridge

Storage	Docker named volumes
Final result	SUCCESS

3. Implementation steps

Step 1 — Check Docker Compose installation

```
docker compose version
```

Actual output:

```
Docker Compose version v5.6.0
```

Docker Compose was already installed, so no additional installation was required.

Step 2 — Check server resources

```
free -h
df -h /
```

Actual results:

Resource	Available
Total RAM	3.7 GiB
Available RAM	2.5 GiB
Total disk	28 GB
Free disk	18 GB

Purpose: Ensure the EC2 server has sufficient resources to run WordPress and MySQL.

Step 3 — Create project directory

mkdir -p /root/docker03-task5

cd /root/docker03-task5

pwd

Actual directory:

/root/docker03-task5

Step 4 — Create environment variables

Created a `.env` file containing the database details and generated passwords.

cat > .env <<EOF

MYSQL_DATABASE=wordpress

MYSQL_USER=wpuser

MYSQL_PASSWORD=$(openssl rand -hex 16)MYSQL_ROOT_PASSWORD =$(openssl rand -hex 24)

EOF

chmod 600 .env

Verified the file:

ls -la .env

Actual output:

-rw------- 1 root root 160 .env

Purpose: Keep database credentials separate from the Compose YAML configuration.

The `.env` file should not be committed to GitHub. For production, a dedicated secrets-management solution is preferable.

4. Create Docker Compose configuration

Step 5 — Create compose.yaml

cat > compose.yaml <<'EOF'

services:

```
  db:
    image: mysql:8.0
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - wordpress_network
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u root
-p\"$$MYSQL_ROOT_PASSWORD\" --silent"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s

  wordpress:
    image: wordpress:php8.3-apache
    restart: unless-stopped
    ports:
      - "8089:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_NAME: ${MYSQL_DATABASE}
      WORDPRESS_DB_USER: ${MYSQL_USER}
      WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD}
    depends_on:
      db:
        condition: service_healthy
```

```
    volumes:
      - wordpress_data:/var/www/html
    networks:
      - wordpress_network

volumes:
  mysql_data:
  wordpress_data:

networks:
  wordpress_network:
    driver: bridge
EOF
```

Explanation of YAML options

Option	Purpose
`services`	Define multiple containers
`image`	Specify the Docker image
`restart`	Configure restart behavior
`environment`	Pass configuration variables
`ports`	Publish WordPress on host port 8089

`depends_on`	Wait for the database health condition
`healthcheck`	Check MySQL health
`volumes`	Preserve application and database files
`networks`	Enable container-to-container communication

5. Validate and deploy application

Step 6 — Validate Docker Compose file

docker compose config --quiet

Actual result: No errors.

Purpose: Validate the Compose configuration before creating containers.

Step 7 — Start WordPress and MySQL

docker compose up -d

Purpose: Create and start services, volumes and the Docker network defined in `compose.yaml`.

The `-d` option runs containers in the background.

Step 8 — Verify running containers

docker compose ps

Actual results:

Container	Status	Port

`docker03-task5-db-1`	Up (healthy)	3306 internal
`docker03-task5-wordpress-1`	Up	8089 → 80

Both services started successfully.

6. Verify application using curl and browser

Step 9 — Check WordPress HTTP response

curl -I http://localhost:8089/

Actual response:

> HTTP/1.1 302 Found
>
> Server: Apache/2.4.68 (Debian)
>
> X-Powered-By: PHP/8.3.35
>
> Location: http://localhost:8089/wp-admin/install.php

Explanation: HTTP 302 indicates WordPress redirected the request to its initial installation page.

Step 10 — Access WordPress in browser

Opened:

> WordPress sample application — Port 8089

The browser displayed the WordPress installation form.

The setup form contained the site title `Docker-03 DevOps Project` and administrator username `devopsadmin`. The completion of the WordPress administrator installation was not separately verified in the supplied output.

7. Verify Docker volumes and networking

Step 11 — Check Docker volumes

docker volume ls

Actual volumes created:

```
docker03-task5_mysql_data
docker03-task5_wordpress_data
```

Purpose: Persist MySQL database files and WordPress application files independently of the containers.

Step 12 — Check Docker networks

```
docker network ls
```

Actual custom network:

```
docker03-task5_wordpress_network
```

Purpose: Enable internal communication between WordPress and MySQL.

Browser / Client

EC2 port 8089

Docker Compose bridge network

WordPress

Port 80

MySQL

Port 3306

WordPress connects to MySQL using service name `db`

Named volumes preserve application and database data

Step 13 — Test database connectivity

Executed:

```
docker compose exec wordpress \
  php -r '$db = new mysqli("db", getenv("WORDPRESS_DB_USER"), getenv("WORDPRESS_DB_PASSWORD"), getenv("WORDPRESS_DB_NAME")); echo $db->connect_error ?: "Database connection SUCCESS\n";'
```

Actual output:

Database connection SUCCESS

This confirms that WordPress can connect to MySQL using the Docker Compose network and database credentials.

8. Important Docker Compose commands

Command	Purpose
`docker compose version`	Check Compose version
`docker compose up -d`	Create and start services
`docker compose ps`	Check service status
`docker compose logs`	View application logs
`docker compose logs -f`	Follow live logs
`docker compose exec`	Execute a command in a service
`docker compose config --quiet`	Validate configuration
`docker compose restart`	Restart services
`docker compose stop`	Stop services

`docker compose down`	Remove containers and project network
`docker volume ls`	List Docker volumes
`docker network ls`	List Docker networks

Important: `docker compose down` normally preserves named volumes. Using `docker compose down -v` also removes the project's named volumes and can permanently delete stored application/database data.

9. Real-time DevOps interview questions

Q1. What is Docker Compose?

Docker Compose is a tool used to define and manage multi-container applications using YAML configuration.

Q2. Why use Docker Compose instead of individual Docker run commands?

It allows us to define services, networking, volumes, environment variables and dependencies in a single configuration, simplifying deployment and maintenance.

Q3. How do WordPress and MySQL communicate?

Both containers use a common Docker bridge network. WordPress connects using the MySQL service name `db` on port 3306.

Q4. Why do we use Docker volumes?

We use volumes to preserve important application and database data even when containers are recreated.

Q5. What is the purpose of `depends_on`?

It controls startup dependencies between services. In this project, it was configured with `condition: service_healthy` so WordPress starts after MySQL passes its health check.

Q6. How do you troubleshoot a failed Compose deployment?

I check service status using `docker compose ps`, review logs using `docker compose logs`, validate configuration, and check networking, credentials and resource availability.

Q7. Why wasn't MySQL port 3306 publicly exposed?

WordPress accesses MySQL through the internal Docker network. Publishing the database port on the EC2 host was unnecessary and would increase its exposure.

Q8. How did you verify your Docker Compose application?

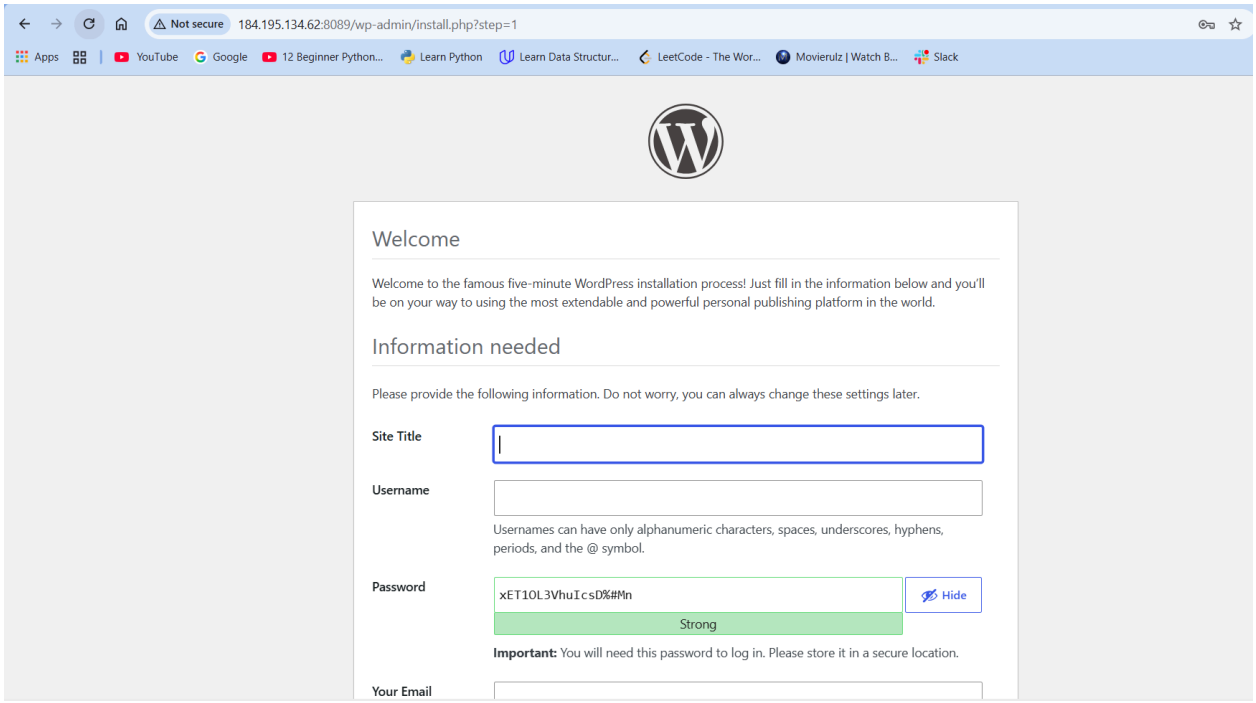
I checked service health using `docker compose ps`, verified the WordPress installation page through the browser, checked Docker volumes and networks, and tested MySQL connectivity from the WordPress container.

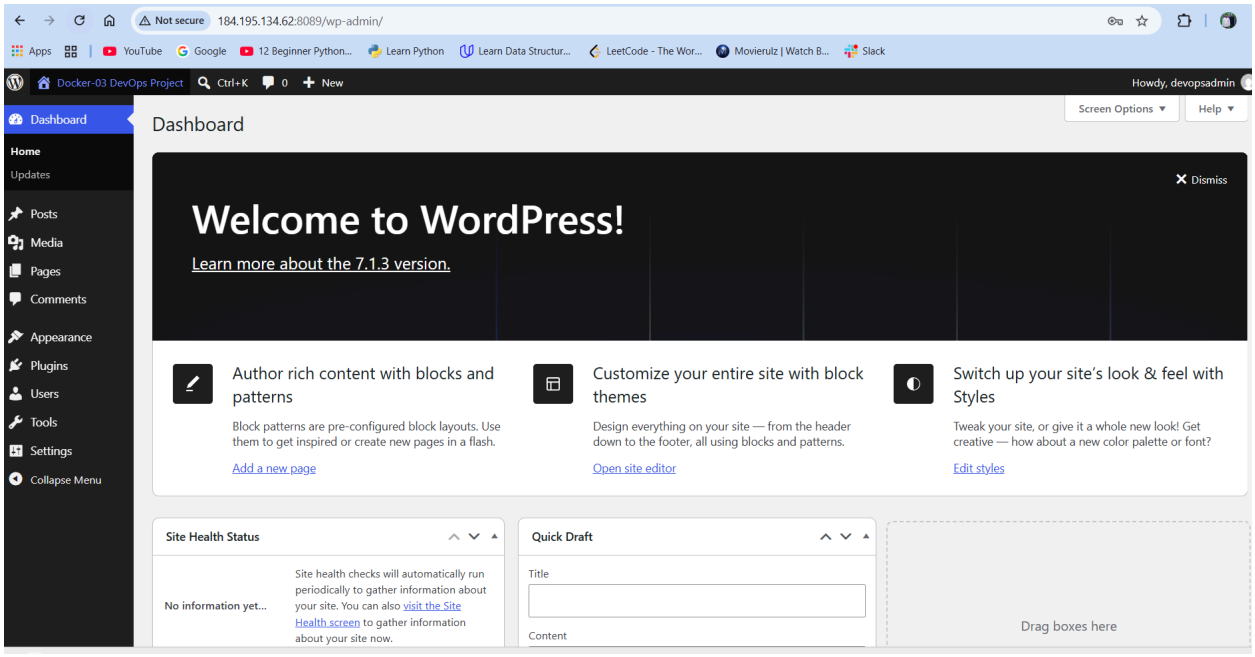
10. Conclusion

Successfully deployed a WordPress and MySQL multi-container application using Docker Compose, configured environment variables, persistent volumes, a custom bridge network and a MySQL health check.

Verified that both containers were running, MySQL was healthy, WordPress was accessible on port 8089 and database connectivity was successful.

Final result: Docker-03 Task 5 — SUCCESS





6. Implement solution to scan images when pushed to docker registry.

1. Objective

Enable Docker Scout image analysis for a Docker Hub repository, push a Docker image, and identify security vulnerabilities in the uploaded image.

Purpose: In a real DevOps environment, Docker images should be checked for known vulnerabilities before being deployed to production.

2. Environment details

Component	Actual configuration
Server	AWS EC2 — Ubuntu
Docker registry	Docker Hub

Security scanner	Docker Scout
Docker Hub username	`chandrashekar8639211489`
Repository	`chandu-nginx`
Original image	`chandu-nginx:v1`
New image tag	`v2`
Image scanning	Active
Scan result	13 High vulnerabilities
Task status	SUCCESS

3. Implementation steps

Step 1 — Open Docker Hub

Opened Docker Hub and signed in.

Navigated to:

My Hub → Repositories → chandu-nginx

Repository:

chandrashekar8639211489/chandu-nginx

The repository was public and initially showed Docker Scout as inactive.

Step 2 — Open repository settings

Inside the repository, clicked Settings and located the section:

Image security insight settings

The available options were:

- Docker Scout image analysis
- None

Selected Docker Scout image analysis.

Step 3 — Attempt to enable scanning

Clicked Save.

Initially, Docker Hub displayed:

Unable to change Image security analytics settings.

The settings page did not confirm successful activation.

Step 4 — Verify scanning from Docker Scout dashboard

Opened:

Docker Scout Repository Settings

Selected the Docker Hub account and checked the `chandu-nginx` repository.

The dashboard showed:

Active Scout repositories: 1

Scout image analysis: Active

Result: Docker Scout scanning was confirmed active.

The earlier settings error did not require further troubleshooting because the Scout dashboard showed that the repository was enabled.

4. Push Docker image to Docker Hub

Step 5 — Tag the existing Docker image

On the Ubuntu EC2 server, executed:

```
sudo docker tag chandu-nginx:v1 \
  chandrashekar8639211489/chandu-nginx:v2
```

Purpose: Assign a new version tag to the existing Docker image.

No image rebuild was required.

Step 6 — Push the image

```
sudo docker push \
  chandrashekar8639211489/chandu-nginx:v2
```

Actual output included:

```
2f042eb1ad72: Layer already exists
8e72e2766ac8: Already exists
44136fa355b3: Already exists
7234d0a9b6ba: Layer already exists
a198c1e4c2c9: Layer already exists

v2: digest: sha256:93d0d21cefc7...
```

Result: Image `v2` successfully pushed to Docker Hub.

The message `Layer already exists` means Docker Hub already stored those image layers and did not need to upload them again.

5. Verify Docker Scout vulnerability report

Step 7 — Open image analysis

Opened the Docker Scout Dashboard.

Navigated to:

Images → chandrashekar8639211489/chandu-nginx

The dashboard displayed image tags `v1` and `v2`, each with vulnerability results.

Step 8 — Review scan findings

Actual Docker Scout scan results

0

Critical

13

High

0

Medium

0

Low

Results shown for both v1 and v2 in your Docker Scout screenshot.

Property	v1	v2
Architecture	linux/amd64	linux/amd64
Critical	0	0
High	13	13
Medium	0	0

Low	0	0

Both tags showed identical findings because they pointed to the same underlying Docker image.

The individual CVE identifiers and affected package versions were not shown in the screenshot, so those details were not verified.

6. Issue faced and resolution

Issue	Observation	Resolution
Scout initially inactive	Repository list showed inactive	Opened repository security settings
Unable to change settings	Docker Hub displayed an error after Save	Checked the dedicated Scout dashboard
Scanner activation verification	Scout dashboard showed Active	Confirmed the repository was enabled
Same vulnerabilities in v1 and v2	Both tags contained 13 High findings	Explained that both tags referenced the same image

7. Important Docker commands

Command	Purpose
`docker images`	Check available local images

`docker tag`	Assign an image registry name and version
`docker login`	Authenticate with Docker Hub
`docker push`	Upload images to Docker Hub
`docker pull`	Download images from Docker Hub
`docker image inspect`	Inspect image metadata

The Docker Scout analysis was enabled and verified through the web dashboard. A local `docker scout cves` command was not part of your verified Task 6 execution.

8. Important DevOps interview questions

Q1. What is Docker Scout?

Docker Scout is a container security tool that analyzes Docker images and reports known vulnerabilities in their packages and dependencies.

Q2. Why do we scan Docker images?

To identify vulnerable packages, assess security risks and address serious findings before applications are deployed.

Q3. How did you implement Docker image scanning?

I enabled Docker Scout image analysis for my Docker Hub repository, pushed an image using `docker push`, and verified the vulnerability findings through the Docker Scout dashboard.

Q4. What is a CVE?

CVE stands for Common Vulnerabilities and Exposures. It provides a standardized identifier for publicly known security vulnerabilities.

Q5. What would you do if a Docker image has High vulnerabilities?

I would inspect the affected packages and available fixes, update the base image or dependencies, rebuild the Docker image, test the application and scan the new image again.

Q6. Does Docker Scout automatically fix vulnerabilities?

No. It identifies vulnerabilities and may provide remediation recommendations. Fixes generally require package updates, dependency changes or rebuilding the image.

Q7. Why did v1 and v2 show the same vulnerabilities?

Because `v2` was created by tagging the same image as `v1`. Changing an image tag does not modify the image contents.

Q8. What is the difference between Docker Hub and Docker Scout?

Docker Hub is a container image registry, while Docker Scout provides security analysis and vulnerability insights for container images.

9. Real-time project explanation

For an interview, you can explain your practical like this:

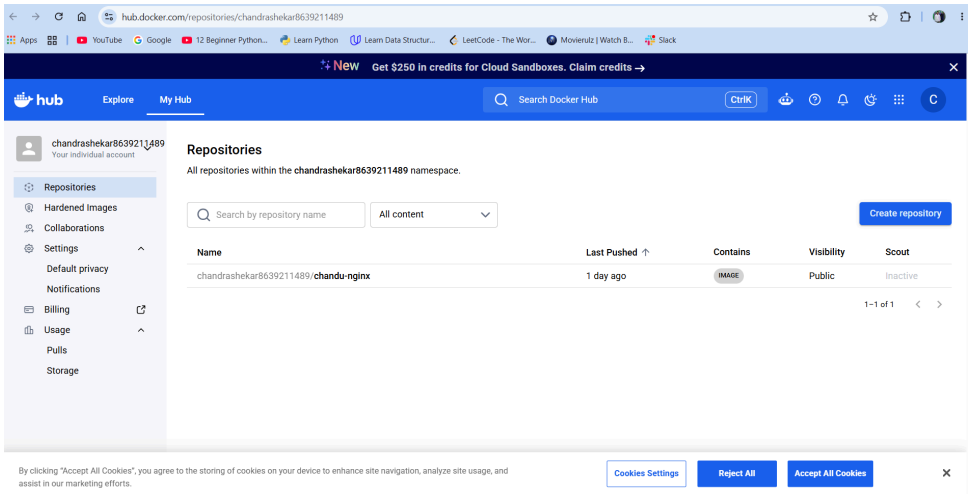
> In my Docker security practical, I configured Docker Scout image analysis for a Docker Hub repository. I tagged an existing custom Nginx Docker image as version v2 and pushed it to Docker Hub. After the image was analyzed, I reviewed the Docker Scout dashboard and found 13 high-severity vulnerabilities. This helped me understand how container image scanning identifies security risks before deployment and how vulnerable packages can be reviewed for remediation.

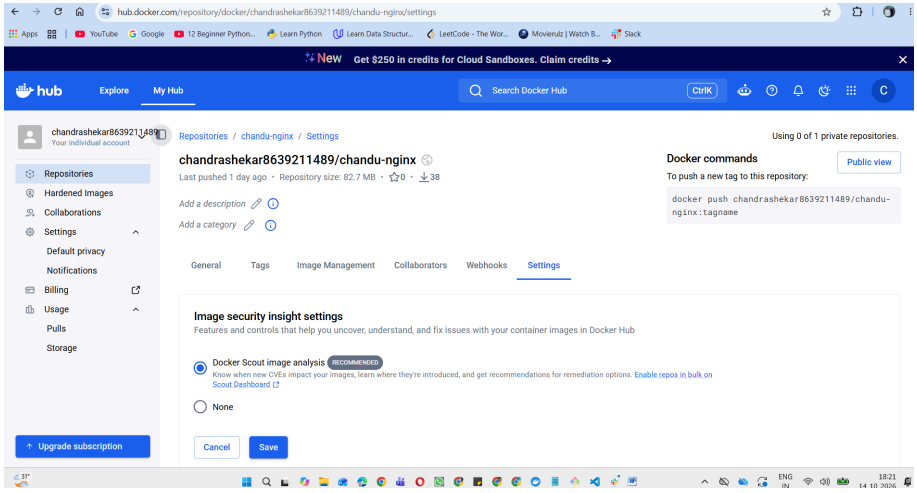
10. Conclusion

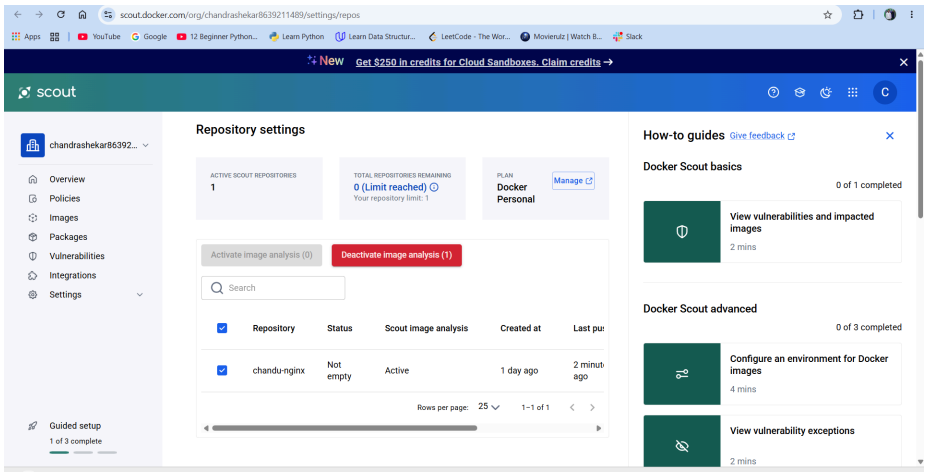
Successfully configured and verified Docker Scout image analysis for the `chandu-nginx` Docker Hub repository.

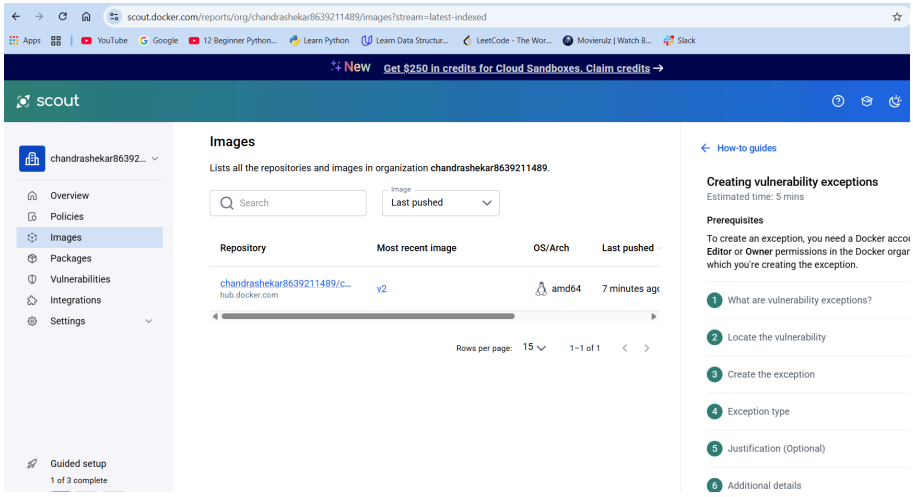
Pushed a new image tag `v2`, reviewed the vulnerability report, and identified 13 High vulnerabilities and zero Critical, Medium or Low findings in the dashboard.

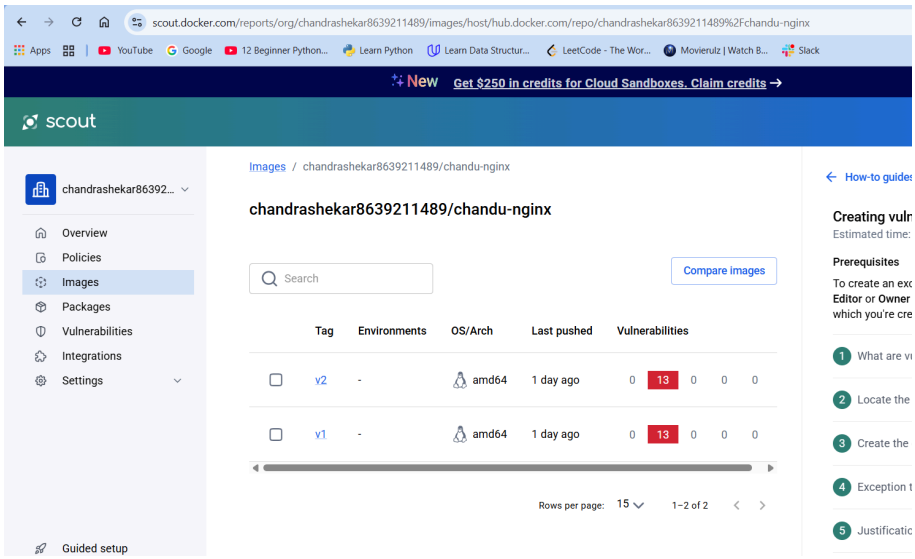
Final Result: Docker-03 Task 6 — SUCCESS


hub.docker.com/repositories/chandrashekar8639211489
New Get $250 in credits for Cloud Sandboxes. Claim credits →
hub
Explore
My Hub
Search Docker Hub
CtrlK
chandrashekar8639211489
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Repositories
All repositories within the chandrashekar8639211489 namespace.
Search by repository name
All content
Create repository
Name
Last Pushed
Contains
Visibility
Scout
chandrashekar8639211489/chandu-nginx
1 day ago
IMAGE
Public
Inactive
1–1 of 1
By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts.
Cookies Settings
Reject All
Accept All Cookies


hub.docker.com/repository/docker/chandrashekar8639211489/chandu-nginx/settings
New Get $250 in credits for Cloud Sandboxes. Claim credits →
hub
Explore
My Hub
Search Docker Hub
CtrlK
chandrashekar8639211489
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / chandu-nginx / Settings
chandrashekar8639211489/chandu-nginx
Last pushed 1 day ago • Repository size: 82.7 MB • ☆0 • ⇩38
Add a description
Add a category
Using 0 of 1 private repositories.
Docker commands
Public view
To push a new tag to this repository:
docker push chandrashekar8639211489/chandu-nginx:tagname
General
Tags
Image Management
Collaborators
Webhooks
Settings
Image security insight settings
Features and controls that help you uncover, understand, and fix issues with your container images in Docker Hub
Docker Scout image analysis
RECOMMENDED
Know when new CVEs impact your images, learn where they're introduced, and get recommendations for remediation options. Enable repos in bulk on Scout Dashboard
None
Cancel
Save


scout.docker.com/org/chandrashekar8639211489/settings/repos
New Get $250 in credits for Cloud Sandboxes. Claim credits →
scout
chandrashekar86392...
Overview
Policies
Images
Packages
Vulnerabilities
Integrations
Settings
Guided setup
1 of 3 complete
Repository settings
ACTIVE SCOUT REPOSITORIES
1
TOTAL REPOSITORIES REMAINING
0 (Limit reached)
Your repository limit: 1
PLAN
Docker Personal
Manage
Activate image analysis (0)
Deactivate image analysis (1)
Search
Repository
Status
Scout image analysis
Created at
Last pu
chandu-nginx
Not empty
Active
1 day ago
2 minut ago
Rows per page: 25
1–1 of 1
How-to guides
Give feedback
Docker Scout basics
0 of 1 completed
View vulnerabilities and impacted images
2 mins
Docker Scout advanced
0 of 3 completed
Configure an environment for Docker images
4 mins
View vulnerability exceptions
2 mins





7. Implement solution to scan images when pushed to AWS ECR.

1. Task objective

Task: Scan Docker images when pushed to Amazon Elastic Container Registry (ECR).

Purpose: Identify known security vulnerabilities in container images before using them for application deployment.

2. Environment details

Component	Configuration

Cloud	AWS
EC2 operating system	Ubuntu
AWS region	`us-east-1`
AWS account	`986881030460`
IAM role	`Docker-ECR-Role`
ECR repository	`chandu-nginx`
Docker image	`chandu-nginx:v3`
Base image	Amazon Linux
Application	Nginx
Scanner	Amazon ECR Basic scanning

3. Workflow

Ubuntu EC2 Server

Build Docker image v3

Push image to AWS ECR

Start ECR Basic scan

Review vulnerability findings

4. Step-by-step practical documentation

Step 1 — Verify the ECR scanning configuration

We executed:

```
aws ecr get-registry-scanning-configuration \
  --region us-east-1
```

Initially, AWS returned:

AccessDeniedException
ecr:GetRegistryScanningConfiguration

Reason: The EC2 IAM role lacked permission to read the registry scanning configuration.

We created an IAM inline policy named `Allow-ECR-Scanning-Read`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadECRScanningConfiguration",
      "Effect": "Allow",
      "Action": [
        "ecr:GetRegistryScanningConfiguration"
      ],
      "Resource": "*"
    }
  ]
}
```

After updating IAM, the command returned:

```
{
  "registryId": "986881030460",
  "scanningConfiguration": {
    "scanType": "BASIC",
    "rules": []
  }
}
```

This showed that Basic scanning was selected, but no scan-on-push rule had been configured.

Step 2 — Enable scan-on-push for the repository

In AWS Console:

1. Open Amazon ECR → Private registry → Features & Settings → Scanning.
2. Select Basic scanning.
3. Leave Scan on push all repositories unchecked.
4. Add the repository filter `chandu-nginx`.
5. Save the configuration.

Verification command:

```
aws ecr get-registry-scanning-configuration \
  --region us-east-1
```

Actual successful output:

```
{
  "registryId": "986881030460",
  "scanningConfiguration": {
    "scanType": "BASIC",
    "rules": [
      {
        "scanFrequency": "SCAN_ON_PUSH",
        "repositoryFilters": [
          {
            "filter": "chandu-nginx",
```

```
          "filterType": "WILDCARD"
        }
      ]
    }
  ]
 }
}
```

Result: ECR scan-on-push was configured for the `chandu-nginx` repository.

Step 3 — Tag and push Docker image v2

We first tagged the existing image:

```
docker tag chandu-nginx:v1 \
  986881030460.dkr.ecr.us-east-1.amazonaws.com/chandu-nginx:v2
```

Authenticated to ECR:

```
aws ecr get-login-password --region us-east-1 | \
docker login \
  --username AWS \
  --password-stdin \
  986881030460.dkr.ecr.us-east-1.amazonaws.com
```

Output:

```
Login Succeeded
```

Pushed the image:

```
docker push \
  986881030460.dkr.ecr.us-east-1.amazonaws.com/chandu-nginx:v2
```

The push succeeded. ECR reported:

```
v2: digest: sha256:93d0d21cefc71351f522fabf60800bd8d1e1deb5a92e9eb7d0020e027a2f753c
```

However, retrieving findings returned `ScanNotFoundException`.

Step 4 — Fix manual scanning permissions

We attempted:

```
aws ecr start-image-scan \
  --repository-name chandu-nginx \
  --image-id imageTag=v2 \
  --region us-east-1
```

AWS returned:

```
AccessDeniedException
ecr:StartImageScan
```

We created another IAM inline policy:

Policy name: `Allow-ECR-Image-Scanning`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowECRImageScanning",
      "Effect": "Allow",
      "Action": [
        "ecr:StartImageScan",
        "ecr:DescribeImageScanFindings"
      ],
      "Resource": "arn:aws:ecr:us-east-1:986881030460:repository/chandu-nginx"
    }
  ]
}
```

This permitted starting manual scans and retrieving findings for our specific ECR repository.

Step 5 — Investigate the unsupported image type

After resolving IAM permissions, AWS returned a different error:

UnsupportedImageTypeException

An artifact with media type

'application/vnd.oci.image.index.v1+json'

cannot be scanned.

Reason: The pushed v2 artifact was an OCI image index rather than an individual image manifest supported by the attempted ECR Basic scan.

We resolved this by building a new Docker image with provenance and SBOM attestations disabled.

Step 6 — Build Docker image v3

We used the existing Docker project:

cd /home/ubuntu/docker-02-task4

Dockerfile:

```
FROM amazonlinux:latest

LABEL maintainer="Chandrashekar"

RUN dnf install -y nginx && dnf clean all

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Build command:

```
docker buildx build \
  --platform linux/amd64 \
  --provenance=false \
  --sbom=false \
```

--load \

-t chandu-nginx:v3 .

Command explanation:

Option	Purpose
`--platform linux/amd64`	Targets AMD64 Linux
`--provenance=false`	Disables provenance attestations
`--sbom=false`	Disables SBOM attestations
`--load`	Loads the built image into the local Docker image store
`-t chandu-nginx:v3`	Assigns the image name and version
`.`	Uses the current directory as the build context

The build completed successfully.

Verification:

docker image ls chandu-nginx

Actual images:

chandu-nginx:v1 93d0d21cefc7

chandu-nginx:v3 cc84ab344610

Step 7 — Push v3 to ECR

Tag the new image:

```
docker tag chandu-nginx:v3 \
  986881030460.dkr.ecr.us-east-1.amazonaws.com/chandu-nginx:v3
```

Push:

```
docker push \
  986881030460.dkr.ecr.us-east-1.amazonaws.com/chandu-nginx:v3
```

Actual digest:

```
sha256:cc84ab344610136265l8adeef8d65c0923fb88f3481bc74bf6f903dbfc13d061
```

Verify image type:

```
aws ecr describe-images \
  --repository-name chandu-nginx \
  --image-ids imageTag=v3 \
  --region us-east-1 \
  --query
'imageDetails[].{Tags:imageTags,MediaType:imageManifestMediaType,ScanStatus:imageScanS
tatus}'
```

Actual output:

```
[
  {
    "Tags": ["v3"],
    "MediaType": "application/vnd.oci.image.manifest.v1+json",
    "ScanStatus": null
  }
]
```

The image was now represented by a single OCI image manifest.

Step 8 — Start ECR Basic scanning

The initial attempt to read scan findings for `v3` returned `ScanNotFoundException`.

We started a manual scan:

```
aws ecr start-image-scan \
  --repository-name chandu-nginx \
  --image-id imageTag=v3 \
  --region us-east-1
```

Actual response:

```
{
  "imageScanStatus": {
    "status": "PENDING"
  }
}
```

`PENDING` meant AWS accepted the scan request and queued the image for processing.

Step 9 — Retrieve vulnerability findings

Final verification command:

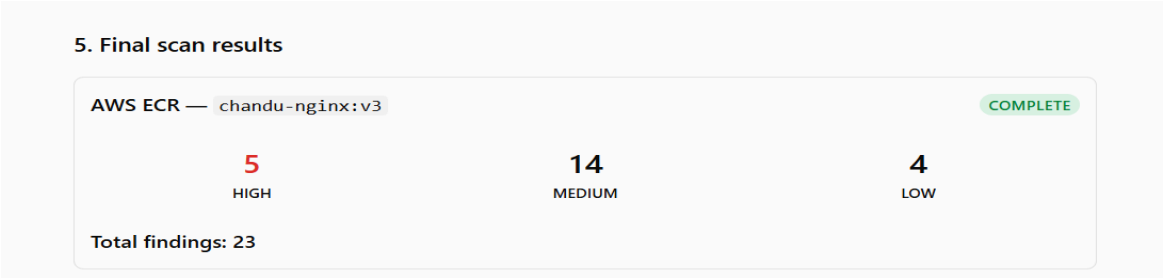
```
aws ecr describe-image-scan-findings \
  --repository-name chandu-nginx \
  --image-id imageTag=v3 \
  --region us-east-1 \
  --query '{Status:imageScanStatus.status,Findings:imageScanFindings.findingSeverityCounts}'
```

Actual successful output:

```
{
  "Status": "COMPLETE",
  "Findings": {
    "HIGH": 5,
    "MEDIUM": 14,
    "LOW": 4
  }
}
```

Result: AWS ECR successfully completed the scan and reported 23 vulnerability findings. The scan was manually initiated; automatic scan-on-push was configured but not independently verified to trigger.

5. Final scan results



The 5 High findings require investigation before production deployment. A completed scan does not mean the image is secure or ready for production.

6. Errors faced and solutions

Error	Root cause	Solution
`AccessDenied` — GetRegistryScanningConfiguration	Missing IAM permission	Added read inline policy
`ScanNotFoundException`	No scan record available	Investigated scanning and image type
`AccessDenied` — StartImageScan	Missing manual-scan permission	Added scanning inline policy
`UnsupportedImageTypeException`	OCI image index not supported by attempted scan	Rebuilt using `--provenance=false --sbom=false`

`ScanNotFoundException` for v3	No scan available after image push	Started a manual scan
`PENDING`	Scan queued for processing	Waited and checked findings
`COMPLETE`	Scan completed	Retrieved vulnerability counts

7. Important commands to remember

Command	Purpose
`docker buildx build`	Build Docker image
`docker tag`	Assign ECR repository tag
`docker push`	Upload image to ECR
`aws ecr get-login-password`	Authenticate Docker to ECR
`aws ecr get-registry-scanning-configuration`	Check registry scanning settings
`aws ecr describe-images`	Verify image details

`aws ecr start-image-scan`	Manually initiate vulnerability scan
`aws ecr describe-image-scan-findings`	Retrieve vulnerability report

8. Real-time interview questions and answers

Q1. What is AWS ECR?

Amazon Elastic Container Registry is a managed AWS service used to store, manage, and retrieve container images.

Q2. Why do we scan Docker images?

To identify known vulnerabilities in operating-system packages and other supported dependencies before deploying containers.

Q3. What is ECR Basic scanning?

It is a built-in ECR vulnerability scanning capability. It supports manual scans and configurable scan-on-push behavior.

Q4. What is the difference between Basic and Enhanced scanning?

Basic scanning offers image vulnerability checks within ECR. Enhanced scanning integrates with Amazon Inspector and provides more extensive and ongoing vulnerability monitoring, with potentially additional charges.

Q5. How did you resolve the unsupported image type error?

The initial image was pushed as an OCI image index, which the attempted Basic scan did not support. I rebuilt a single-platform image without provenance and SBOM attestations, pushed it with a new tag, and successfully scanned it.

Q6. What do you do when a Docker image contains High vulnerabilities?

I identify the affected packages and CVEs, check available fixes, update the base image or dependencies, rebuild the image, and scan it again. For production, I follow the organization's security acceptance policy.

Q7. How would you automate scanning in a CI/CD pipeline?

I would configure image scanning in the container registry, build and push images through Jenkins, check the scan findings, and use a defined security threshold to decide whether deployment can proceed.

9. Project explanation for an interview

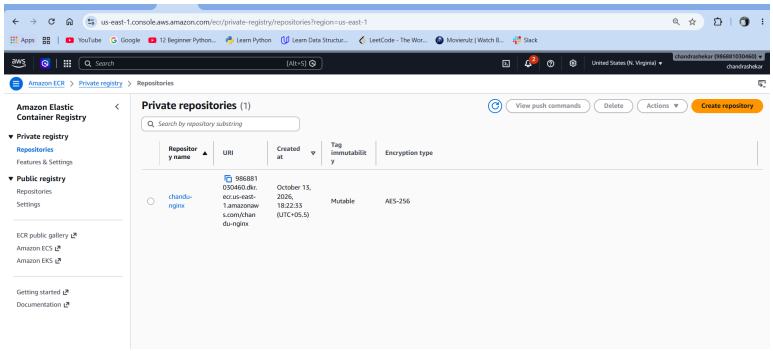
In my Docker project, I integrated AWS ECR Basic image scanning to identify security vulnerabilities in container images. I configured scan-on-push settings for a private ECR repository, updated the EC2 IAM role with required permissions, and pushed a custom Nginx Docker image.

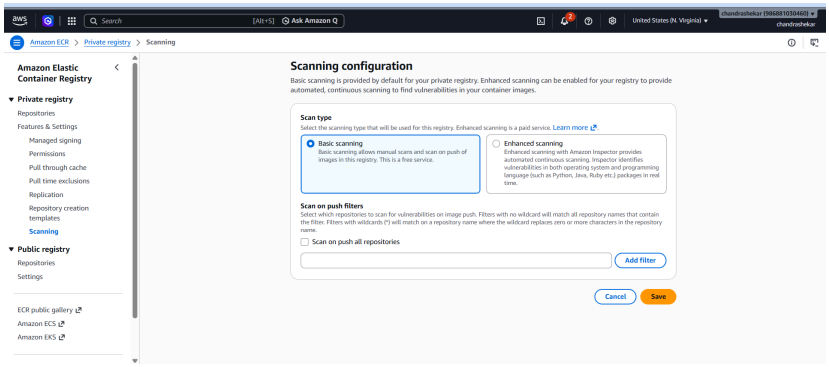
During implementation, I faced an unsupported OCI image index issue. I resolved it by rebuilding the image with Buildx without provenance and SBOM attestations.

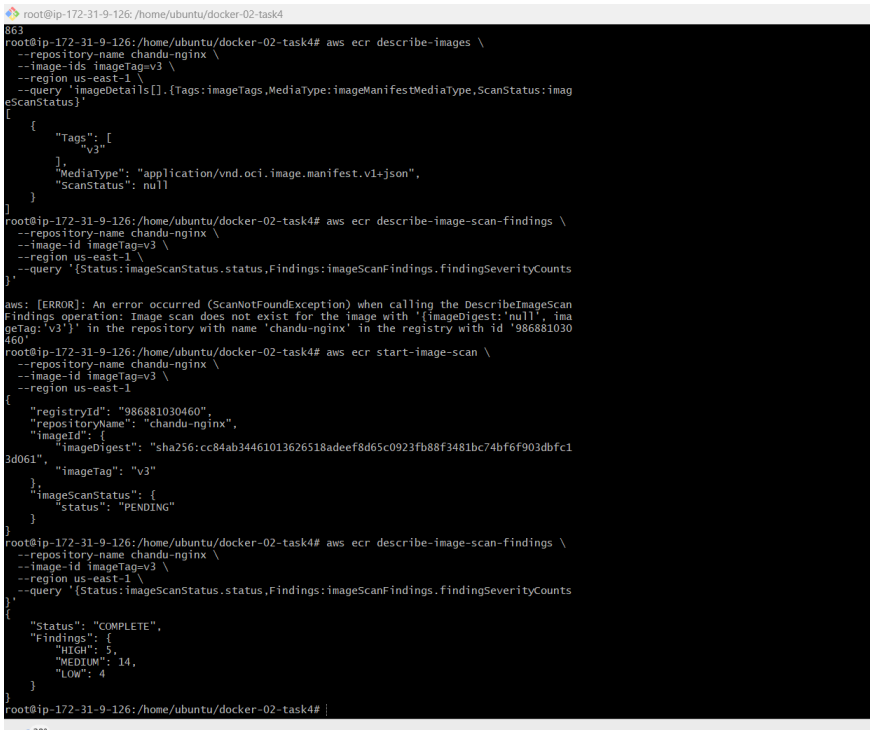
I then successfully performed a manual ECR image scan. The scan completed and identified 5 High, 14 Medium, and 4 Low vulnerability findings.

10. Conclusion

Docker-03 Task 7 — Completed







8. Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

objective

Task: Create a Jenkins Declarative Pipeline to automatically build a Docker image and push it to Docker Hub.

Purpose: Automate container image creation and publishing using GitHub, Jenkins, Docker, and Docker Hub instead of manually running every command.

2. Environment details

Component	Actual configuration
Cloud platform	AWS EC2
Operating system	Ubuntu

Jenkins	2.580.1
Jenkins installation	Docker container
Jenkins container	`jenkins-container`
Jenkins port	`8080`
Jenkins agent	`Docker03-Agent`
Agent label	`docker03`
Agent username	`jenkinsagent`
Java	21.0.12.1
Docker	29.9.0
GitHub branch	`main`
Docker Hub username	`chandrashekar8639211489`
Docker Hub credentials ID	`dockerhub-creds`

Pipeline job	`Docker03-Task08-DockerHub-Pipeline`

3. Architecture

GitHub Repository

Dockerfile + index.html

Jenkins Controller

Runs inside Docker on port 8080

SSH connection

Docker03-Agent

Ubuntu EC2 + Java + Docker

Docker Build

Builds custom Nginx image

Docker Hub

chandu-nginx:build-2

4. Practical steps

Step 1 — Start the existing Jenkins container

First, we checked all Docker containers:

docker ps -a

We found `jenkins-container` was stopped.

Started Jenkins:

docker start jenkins-container

Verified:

docker ps --filter "name=jenkins-container"

docker logs --tail 30 jenkins-container

Actual result:

Jenkins is fully up and running

Purpose: Restart the existing Jenkins container without reinstalling Jenkins or losing previously configured jobs.

Step 2 — Verify Jenkins installation and storage

Checked whether Docker CLI existed inside the controller:

docker exec jenkins-container docker --version

Error:

exec: "docker": executable file not found in $PATH

Checked Jenkins mounts:

```
docker inspect jenkins-container \
  --format '{{json .Mounts}}'
```

Actual result showed a persistent Docker volume:

jenkins_home → /var/jenkins_home

Decision: Use a dedicated Ubuntu build agent instead of directly granting Docker access to the Jenkins controller.

Step 3 — Install Java 21 on Ubuntu

Checked Java:

java -version

Initially, Java was not installed.

Installation commands:

apt update

apt install -y openjdk-21-jre-headless

java -version

Actual result:

openjdk version "21.0.12.1"

Purpose: Jenkins agents require a supported Java runtime to communicate with the controller.

Step 4 — Create a Jenkins agent user

Created a dedicated user:

useradd -m -s /bin/bash jenkinsagent

Created its workspace:

mkdir -p /home/jenkinsagent/agent
chown -R jenkinsagent:jenkinsagent /home/jenkinsagent/agent

Verified:

id jenkinsagent
ls -ld /home/jenkinsagent/agent

Actual user ID:

uid=1001(jenkinsagent)
gid=1001(jenkinsagent)

Purpose: Jenkins build jobs run under a dedicated Linux user instead of running directly as root.

Step 5 — Verify SSH service

Checked SSH:

systemctl status ssh --no-pager

Checked port 22:

ss -tlnp | grep ':22 '

Actual result:

Active: active (running)
0.0.0.0:22 LISTEN

Purpose: The Jenkins controller uses SSH to launch the agent on Ubuntu.

Step 6 — Generate the SSH key pair

On the EC2 host, ran:

```
ssh-keygen -t ed25519 \
  -f /root/jenkins-agent-key \
  -N "" \
  -C "jenkins-docker03-agent"
```

Configured public-key authorization:

```
install -d -m 700 \
  -o jenkinsagent -g jenkinsagent \
  /home/jenkinsagent/.ssh

cat /root/jenkins-agent-key.pub >> \
  /home/jenkinsagent/.ssh/authorized_keys

chown jenkinsagent:jenkinsagent \
  /home/jenkinsagent/.ssh/authorized_keys

chmod 600 /home/jenkinsagent/.ssh/authorized_keys
```

Tested SSH:

```
ssh -i /root/jenkins-agent-key \
  -o StrictHostKeyChecking=accept-new \
  jenkinsagent@127.0.0.1 \
  'whoami && java -version'
```

Actual output:

```
jenkinsagent
openjdk version "21.0.12.1"
```

Purpose: Enable SSH key-based authentication without using a login password.

Step 7 — Verify communication between Jenkins and Ubuntu

Because Jenkins runs inside a container, its `127.0.0.1` address is different from Ubuntu's localhost.

We checked the Docker network route:

```
docker exec jenkins-container \
  bash -c 'cat /proc/net/route'
```

Identified the Docker gateway:

172.17.0.1

Tested the SSH connection:

```
docker exec jenkins-container \
  bash -c 'timeout 5 bash -c "</dev/tcp/172.17.0.1/22" && echo "SSH CONNECTION SUCCESS" || echo "SSH CONNECTION FAILED"'
```

Actual output:

SSH CONNECTION SUCCESS

Step 8 — Configure SSH credentials in Jenkins

Navigated to:

Manage Jenkins → Credentials → System → Global credentials → Add Credentials

Selected:

Setting	Value
Kind	SSH Username with private key
Username	`jenkinsagent`
Private key	From `/root/jenkins-agent-key`

Description	Docker-03 Ubuntu SSH Agent

The saved credential had an automatically generated ID and was successfully used by Jenkins.

The private key must remain secret. Do not add it to GitHub or documentation.

Step 9 — Create Docker03-Agent in Jenkins

Navigated to:

Manage Jenkins → Nodes → New Node

Configured:

Setting	Value
Node name	`Docker03-Agent`
Node type	Permanent Agent
Executors	1
Remote root directory	`/home/jenkinsagent/agent`
Labels	`docker03`
Usage	Only jobs matching label
Launch method	Launch agents via SSH

Host	`172.17.0.1`
Port	`22`
Credentials	`jenkinsagent`
Host verification	Known hosts file verification

Step 10 — Resolve Jenkins SSH host-key error

The agent initially failed to connect.

Error:

No Known Hosts file was found at

/var/jenkins_home/.ssh/known_hosts

Created the SSH directory inside Jenkins:

```
docker exec -u root jenkins-container \
  bash -c 'mkdir -p /var/jenkins_home/.ssh && chown jenkins:jenkins /var/jenkins_home/.ssh && chmod 700 /var/jenkins_home/.ssh'
```

Added the host's public key:

```
HOST_KEY=$(cat /etc/ssh/ssh_host_ed25519_key.pub)


printf '172.17.0.1 %s\n' "$HOST_KEY" | \
docker exec -i -u jenkins jenkins-container \
sh -c 'cat >> /var/jenkins_home/.ssh/known_hosts && chmod 600 /var/jenkins_home/.ssh/known_hosts'
```

The SSH hostname was corrected from `172.31.9.126` to `172.17.0.1` to match the verified connection and known-hosts entry.

The successful Jenkins log reported:

[SSH] SSH host key matches key in Known Hosts file.

[SSH] Authentication successful.

Agent successfully connected and online

Step 11 — Configure Docker permissions on the Jenkins agent

The Docker CLI was installed, but the agent user initially received:

permission denied while trying to connect to

unix:///var/run/docker.sock

Added the agent user to the Docker group:

usermod -aG docker jenkinsagent

Verified membership:

id jenkinsagent

Actual result:

groups=1001(jenkinsagent),986(docker)

Tested:

sudo -u jenkinsagent -H docker ps

The command successfully listed the running Docker containers.

We also reconnected the Jenkins agent so its new group membership was available to Jenkins.

Security note: The Docker group effectively grants root-level privileges on the host. This arrangement was used for a trusted practice lab; production systems should use isolated build agents and stronger privilege boundaries.

Step 12 — Create the Jenkins pipeline job

Created a Jenkins job:

`Docker03-Task08-DockerHub-Pipeline`

Selected Pipeline as the job type.

Initially, we used a test pipeline to verify the Jenkins agent.

```
pipeline {
    agent {
        label 'docker03'
    }

    stages {
        stage('Check Agent') {
            steps {
                sh 'whoami'
                sh 'java -version'
            }
        }

        stage('Check Docker') {
            steps {
                sh 'docker --version'
                sh 'docker info --format "{{.ServerVersion}}"'
            }
        }
    }
}
```

Build #1 result: SUCCESS

Actual output confirmed:

Running on Docker03-Agent
jenkinsagent
openjdk version "21.0.12.1"
Docker version 29.9.0
Finished: SUCCESS

Step 13 — Configure Docker Hub credentials

Navigated to Jenkins Global Credentials.

Created:

Setting	Value
Kind	Username with password
Username	`chandrashekar8639211489`
Password	Docker Hub personal access token
ID	`dockerhub-creds`
Description	Docker Hub credentials for Task 8

Purpose: Let Jenkins authenticate to Docker Hub without hardcoding the token in the pipeline.

Step 14 — Prepare the Docker project

The original Docker project contained an AWS CLI installer archive and extracted installer files, which were unnecessary for this task.

Created a clean project directory:

```
mkdir -p /home/ubuntu/docker03-task8
```

Copied the required files:

```
cp /home/ubuntu/docker-02-task4/Dockerfile \
  /home/ubuntu/docker03-task8/

cp /home/ubuntu/docker-02-task4/index.html \
  /home/ubuntu/docker03-task8/
```

Created `.dockerignore`:

```
cat > /home/ubuntu/docker03-task8/.dockerignore <<'EOF'
.git
.gitignore
Jenkinsfile
*.zip
aws/
EOF
```

The project contained:

```
docker03-task8/
├── Dockerfile
├── index.html
└── .dockerignore
```

Dockerfile:

```
FROM amazonlinux:latest

LABEL maintainer="Chandrashekar"

RUN dnf install -y nginx && dnf clean all

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 15 — Create GitHub repository and push source

Created the public GitHub repository:

[chandrashekar0532/docker03-task8-nginx]

We initially ran `git init` from `/home/ubuntu` by mistake. Git showed unrelated home-directory files such as `.aws/` and `.ssh/`.

Solution: Removed only the accidental `/home/ubuntu/.git` metadata and initialized Git from the correct project directory. No private files were committed.

Correct commands:

cd /home/ubuntu/docker03-task8

git init
git branch -M main

git remote add origin \
  https://github.com/chandrashekar0532/docker03-task8-nginx.git

git add Dockerfile index.html .dockerignore

git commit -m "Add Nginx Docker project for Jenkins pipeline"

Actual commit:

8e27bf5 Add Nginx Docker project for Jenkins pipeline
3 files changed

Pushed the code:

git push -u origin main

Authenticated with a GitHub personal access token at the password prompt.

Actual result:

[new branch] main -> main
branch 'main' set up to track 'origin/main'

5. Final Jenkins Declarative Pipeline

The following pipeline is the script you executed successfully in Build #2.

```
pipeline {
    agent {
        label 'docker03'
    }

    environment {
        DOCKER_IMAGE = 'chandrashekar8639211489/chandu-nginx'
        IMAGE_TAG = "build-${BUILD_NUMBER}"
    }

    stages {
        stage('Git Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/chandrashekar0532/docker03-task8-nginx.git'
            }
        }

        stage('Docker Build') {
            steps {
                sh 'docker build -t DOCKER_IMAGE:{IMAGE_TAG} .'
            }
        }

        stage('Docker Hub Login') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_TOKEN'
                )]) {
                    sh '''
                        echo "$DOCKER_TOKEN" | docker login \
```

```
                --username "$DOCKER_USER" \
                --password-stdin
            '''
        }
    }
}

stage('Docker Push') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-creds',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_TOKEN'
        )]) {
            sh '''
                echo "$DOCKER_TOKEN" | docker login \
                 --username "$DOCKER_USER" \
                 --password-stdin

                docker push DOCKER_IMAGE:{IMAGE_TAG}
            '''
        }
    }
}

stage('Verify Image') {
    steps {
        sh 'docker image inspect DOCKER_IMAGE:{IMAGE_TAG} --format "{{.Id}}"'
        echo 'Docker image built and pushed successfully!'
    }
}
}
```

```
    post {
        always {
            sh 'docker logout || true'
        }

        success {
            echo 'SUCCESS: Docker image published to Docker Hub!'
        }

        failure {
            echo 'FAILED: Check Jenkins console output.'
        }
    }
}
```

6. Pipeline stages and purposes

Stage	Purpose
Git Checkout	Download the application source from GitHub
Docker Build	Build a custom Nginx Docker image
Docker Hub Login	Authenticate using Jenkins credentials
Docker Push	Publish the image to Docker Hub
Verify Image	Confirm the built image exists locally

Post actions	Log out of Docker Hub and report the build result

The tag `build-${BUILD_NUMBER}` ensures that each Jenkins build uses its own image tag.

7. Build #2 — Actual successful output

Docker03-Task08-DockerHub-Pipeline

SUCCESS

Git Checkout

Docker Build

Docker Hub Login

Docker Push

Verify Image

Post Actions

Key console results:

Running on Docker03-Agent

Checking out Revision 8e27bf5...

docker build -t chandrashekar8639211489/chandu-nginx:build-2 .

Login Succeeded

docker push chandrashekar8639211489/chandu-nginx:build-2

build-2: digest:
sha256:906284dd1323266598bd2bfcf7a60e1bd588c2c7a0a546691841229d23c49f52

Docker image built and pushed successfully!

SUCCESS: Docker image published to Docker Hub!

Finished: SUCCESS

8. Docker Hub verification

You opened Docker Hub and confirmed that the new `build-2` tag was published.

View Docker Hub repository tags

Detail	Actual value
Image	`chandrashekar8639211489/chandu-nginx`
Tag	`build-2`
Architecture	linux/amd64
Compressed size	82.74 MB
Docker Hub vulnerability findings	13 High
Image publication	Verified

Docker Hub displayed the new image successfully. The security findings require investigation before production use.

9. Errors encountered and solutions

Error	Cause	Solution
`docker: executable file not found`	Docker CLI missing inside Jenkins controller	Used an Ubuntu SSH build agent
Java command not found	Java missing on Ubuntu host	Installed Java 21
Agent `known_hosts` missing	SSH host not trusted by Jenkins	Added verified SSH host public key
Agent remained offline	Incorrect host setting / SSH verification	Used `172.17.0.1` and relaunched agent
Docker socket permission denied	Agent user lacked Docker daemon access	Added `jenkinsagent` to Docker group and reconnected
`fatal: pathspec 'Dockerfile' did not match`	Git initialized in the wrong directory	Initialized Git in `/home/ubuntu/docker03-task8`
GitHub requested credentials	HTTPS push required authentication	Used a GitHub PAT
Docker credential warning	Docker saved login credentials in agent home	Used `docker logout`; temporary Docker config is recommended for production

10. Jenkins commands and concepts to remember

Command / concept	Explanation
`docker ps -a`	Shows all running and stopped containers
`docker start`	Starts a stopped container
`docker logs`	Displays container logs
`useradd -m`	Creates a Linux user with a home directory
`ssh-keygen`	Generates SSH keys
`authorized_keys`	Specifies which public keys may log in
`known_hosts`	Stores trusted SSH server identities
`usermod -aG docker`	Adds user to Docker group
`git init`	Initializes a Git repository
`git remote add origin`	Connects local repository to GitHub
`git push`	Uploads commits to remote repository

`agent { label 'docker03' }`	Runs Jenkins stages on the named agent
`withCredentials`	Makes Jenkins secrets available to a controlled step
`docker build`	Builds an image from a Dockerfile
`docker login`	Authenticates to Docker Hub
`docker push`	Publishes the image
`post { ... }`	Executes cleanup and success/failure actions

11. Real-time DevOps interview questions

Q1. Explain your Jenkins Docker CI/CD project.

I implemented a Jenkins Declarative Pipeline that checks out Nginx source code from GitHub, builds a Docker image on a dedicated Ubuntu build agent, authenticates to Docker Hub using Jenkins credentials, and automatically pushes the image with the Jenkins build number as its tag.

Q2. Why did you use a Jenkins agent?

The Jenkins controller was running inside a container without Docker CLI or access to the host Docker daemon. I configured a separate Ubuntu SSH agent to execute Docker build and push commands.

Q3. How does the Jenkins controller communicate with the agent?

Through an SSH connection using private-key credentials. The remote machine needs a compatible Java runtime to launch the Jenkins remoting agent.

Q4. Why do we use `known_hosts`?

The file contains trusted SSH server public keys. It helps Jenkins verify that it is connecting to the intended SSH server rather than an impersonator.

Q5. How do you securely store Docker Hub credentials?

Using Jenkins Credentials. The pipeline retrieves them through `withCredentials` rather than hardcoding tokens in source code.

Q6. Why do we use a unique Docker image tag for each build?

A build-number-based tag makes images easier to track, troubleshoot, compare, and deploy independently.

Q7. What happens if the Docker image build fails?

The shell step returns a nonzero exit status. Jenkins marks the stage and build as failed and does not continue through later stages by default.

Q8. How would you improve this pipeline for production?

I would use ephemeral build agents, isolate Docker access, scan images for vulnerabilities, use temporary credentials, enforce security thresholds, pin base image versions or digests, and add application tests before publishing or deploying.

Q9. How can Jenkins automatically start after a GitHub commit?

By configuring a GitHub webhook and a compatible Jenkins trigger so pushes to the configured branch start a pipeline build. In this practical, we ran the pipeline manually; webhook triggering was not configured.

Q10. What is the difference between Docker build and Docker push?

`docker build` creates an image from application files and a Dockerfile. `docker push` uploads the tagged image to a registry such as Docker Hub.

12. Interview project explanation

> I completed a Jenkins CI/CD automation project using AWS EC2, Ubuntu, GitHub, Docker, and Docker Hub.
>
> I installed Java 21 on the Ubuntu host and configured a dedicated Jenkins agent using SSH key-based authentication. I resolved SSH host-key verification and Docker socket permission issues, then verified the agent using a test Jenkins pipeline.

After that, I created a GitHub repository containing an Nginx Dockerfile and HTML application, configured Docker Hub credentials securely in Jenkins, and implemented a Declarative Pipeline.

The pipeline performs Git checkout, Docker image build, Docker Hub authentication, Docker image push, and verification. My second Jenkins build completed successfully, and I confirmed that the `build-2` Docker image was available in Docker Hub.

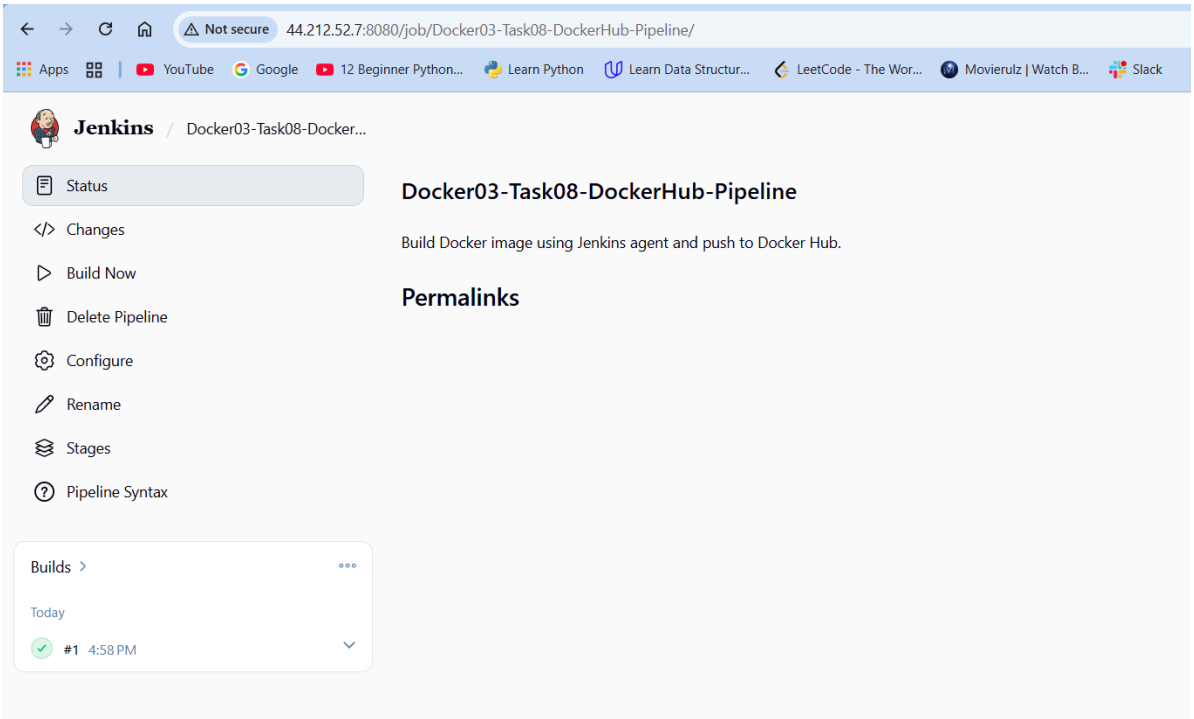
13. Final conclusion

Docker-03 Task 8 successfully completed

You successfully created a Jenkins controller–agent setup and implemented a working CI/CD pipeline to publish Docker images automatically.

Your practical demonstrated hands-on experience with Jenkins Declarative Pipelines, Docker, GitHub, SSH authentication, Jenkins Credentials, Linux permissions, and Docker Hub.

Final result: Jenkins Build #2 — `SUCCESS`; Docker Hub image `chandrashekar8639211489/chandu-nginx:build-2` — published and verified.



ubuntu@ip-172-31-9-126: ~/docker03-task8

```
-rw-r--r-- 1 ubuntu ubuntu  255 Oct 14 17:05 index.html
ubuntu@ip-172-31-9-126:~/docker03-task8$ git init
git branch -M main
git remote add origin https://github.com/chandrashekar0532/docker03-task8-nginx.git
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/ubuntu/docker03-task8/.git/
ubuntu@ip-172-31-9-126:~/docker03-task8$ git add Dockerfile index.html .dockerignore
git commit -m "Add Nginx Docker project for Jenkins pipeline"
git status
[main (root-commit) 8e27bf5] Add Nginx Docker project for Jenkins pipeline
 Committer: Ubuntu <ubuntu@ip-172-31-9-126.ec2.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 3 files changed, 28 insertions(+)
 create mode 100644 .dockerignore
 create mode 100644 Dockerfile
 create mode 100644 index.html
On branch main
nothing to commit, working tree clean
ubuntu@ip-172-31-9-126:~/docker03-task8$ git config user.name "chandrashekar0532"
ubuntu@ip-172-31-9-126:~/docker03-task8$ git remote -v
origin  https://github.com/chandrashekar0532/docker03-task8-nginx.git (fetch)
origin  https://github.com/chandrashekar0532/docker03-task8-nginx.git (push)
ubuntu@ip-172-31-9-126:~/docker03-task8$ git push -u origin main
Username for 'https://github.com': chandrashekar0532
Password for 'https://chandrashekar0532@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 707 bytes | 707.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/chandrashekar0532/docker03-task8-nginx.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
ubuntu@ip-172-31-9-126:~/docker03-task8$
```

